A ≥ B: An LCF-Style Sum-of-Squares Prover for Polynomial Inequalities

Mathematical Foundations and System Design

Luiz Akabane

August 16, 2026

Abstract

A ≥ B proves polynomial inequalities $A \geq B$ by rewriting them as $p = A - B \geq 0$ and certifying $p \geq 0$ with a *sum-of-squares* decomposition $p = \sum_i c_i\, q_i^2$ ($c_i \in \mathbb{Q}_{\geq 0}$, $q_i \in \mathbb{Q}[x_1, \ldots, x_n]$), entirely in exact rational arithmetic. It is built in the LCF tradition: an *untrusted* search engine proposes certificates, and a small, *trusted checker* is the sole authority on whether an inequality is proved, so a bug in the search can lose a proof, but never fake one. This document builds up the mathematics from the ground (nonnegativity and the gap between it and the SOS property, the Gram-matrix and semidefinite reformulation, *Newton-polytope basis reduction*, and exact rational extraction by $LDL^\top$) and then the system that implements it: the exact representations, the trusted checker, the search, denominator clearing, the structured result layer behind its command-line and local web interfaces, and a benchmark corpus with its soundness gate. It also covers the two directions the project has since grown in: *constrained* inequalities through a fragment of the Positivstellensatz, and a checker whose soundness, together with the individual proofs it licenses, is machine-verified in Lean 4.

Contents

1 Introduction

Deciding whether a multivariate polynomial is nonnegative on all of $\mathbb{R}^n$ is NP-hard in general, so any tool that wants to be both automatic and trustworthy has to give something up somewhere. The classical move is to ask for less: rather than test nonnegativity head-on, look for a *certificate* of it in the shape of a sum of squares. Two things make that trade worth it. A sum of squares with nonnegative coefficients is nonnegative for a reason you can see at a glance (no oracle, no leap of faith), and the search for one collapses into *semidefinite programming*, a convex problem we know how to solve.

A $\geq$ B applies this idea to the concrete task of proving inequalities such as

$$a^2 + b^2 + c^2 \;\geq\; ab + bc + ca,$$

by exhibiting and verifying an explicit algebraic identity, here

$$a^2 + b^2 + c^2 - ab - bc - ca \;=\; \tfrac{1}{2}(a-b)^2 + \tfrac{1}{2}(b-c)^2 + \tfrac{1}{2}(a-c)^2,$$

whose right-hand side is visibly nonnegative.

The organising principle is borrowed from the LCF family of proof assistants: keep an *untrusted engine* and a *trusted kernel* strictly apart. The engine is free to be as clever, heuristic, or numerical as it likes; whatever it comes up with is re-verified, exactly, by a small checker, and the program never prints PROVED unless that checker accepts. The payoff is a comforting asymmetry: a bug anywhere in the complicated search can only cost us a proof, not manufacture a wrong one. Driving that search is a *Newton-polytope reduction* that pins down the monomial basis of the SOS problem canonically, and as small as the mathematics allows (Section 5.2).

Section 2 builds the mathematics; Sections 3–6 describe the system and how it clears denominators; Section 7 extends it to inequalities that hold only under side constraints; Section 8 lays out the architecture and Section 9 the validation corpus; and Section 10 covers the formal verification, in Lean 4, of the checker (and of the individual proofs it licenses) before Section 11 sketches what is left.

2 Mathematical foundations

2.1 Polynomials and nonnegativity

Fix variables $\mathbf{x} = (x_1, \ldots, x_n)$. We work in the polynomial ring $\mathbb{Q}[\mathbf{x}]$ over the rationals. A *monomial* is a product $\mathbf{x}^\alpha = x_1^{\alpha_1} \cdots x_n^{\alpha_n}$ indexed by an exponent vector $\alpha \in \mathbb{N}^n$; its *degree* is

$|\alpha| = \sum_i \alpha_i$. A polynomial $p = \sum_\alpha c_\alpha \mathbf{x}^\alpha$ has finitely many nonzero coefficients $c_\alpha \in \mathbb{Q}$, and $\deg p = \max\{\, |\alpha| : c_\alpha \neq 0 \,\}$. Every polynomial defines a function $\mathbb{R}^n \to \mathbb{R}$ by evaluation.

Definition 2.1 (Nonnegative polynomial)**.** $p \in \mathbb{Q}[\mathbf{x}]$ is *nonnegative*, written $p \geq 0$, if $p(\mathbf{x}) \geq 0$ for all $\mathbf{x} \in \mathbb{R}^n$.

2.2 Sum-of-squares certificates

Definition 2.2 (Sum of squares)**.** $p \in \mathbb{Q}[\mathbf{x}]$ is a *sum of squares* (SOS) if there exist rationals $c_1, \ldots, c_m \geq 0$ and polynomials $q_1, \ldots, q_m \in \mathbb{Q}[\mathbf{x}]$ with

$$p \;=\; \sum_{i=1}^{m} c_i \, q_i^2. \tag{1}$$

We call the list $\big((c_i, q_i)\big)_{i=1}^{m}$ an *SOS certificate* for p.

The whole system rests on the following elementary but decisive fact.

Theorem 2.3 (Soundness of SOS certificates)**.** *If p admits an SOS certificate (1), then $p \geq 0$.*

Proof. For every $\mathbf{x} \in \mathbb{R}^n$ and each i, $q_i(\mathbf{x})^2 \geq 0$ because it is the square of a real number, and $c_i \geq 0$; hence $c_i \, q_i(\mathbf{x})^2 \geq 0$. A finite sum of nonnegative reals is nonnegative, so $p(\mathbf{x}) = \sum_i c_i \, q_i(\mathbf{x})^2 \geq 0$. □

Theorem 2.3 is an *implication*. Its converse is false: not every nonnegative polynomial is a sum of squares.

Example 2.4 (Motzkin polynomial)**.** The polynomial

$$M(x, y) \;=\; x^4y^2 + x^2y^4 + 1 - 3x^2y^2$$

satisfies $M \geq 0$ on $\mathbb{R}^2$ (by the AM-GM inequality applied to x^4y^2, x^2y^4, and 1), yet M is *not* a sum of squares. It is the classical witness that nonnegativity is strictly weaker than the SOS property.

The exact boundary is Hilbert's theorem.

Theorem 2.5 (Hilbert, 1888)**.** *Every nonnegative real form (homogeneous polynomial) of degree $2d$ in n variables is a sum of squares if and only if $n \leq 2$, or $2d = 2$, or $(n, 2d) = (3, 4)$. In all other cases there exist nonnegative forms that are not sums of squares.*

Two consequences shape the design.

- **Degree ≤ 2 is complete.** For quadratic p, nonnegativity and the SOS property coincide (Proposition 5.5); the search can never fail on a true quadratic inequality.
- **Higher degree is incomplete.** From degree 6 in 3 variables (Example 2.4) onward, some true inequalities have no SOS certificate. The prover must therefore report a third outcome, NO_CERT_FOUND, which is *not* a disproof.

2.3 The reduction

An inequality $A \geq B$ with $A, B \in \mathbb{Q}[\mathbf{x}]$ is equivalent to $A - B \geq 0$. Setting $p := A - B$ (or $p := B - A$ for $A \leq B$) reduces every claim to the single question: *does p admit an SOS certificate?* The rest of the paper answers this constructively and verifies the answer exactly.

3 Exact representation

Because the checker must decide the polynomial identity (1) *exactly*, every object is represented over $\mathbb{Q}$ with no floating point.

Rationals.
Arbitrary-precision $\mathbb{Q}$ (via a bignum library). All coefficient arithmetic is exact; sign tests and equality are decidable.

Monomials.
Exponent vectors, e.g. $a^2bc^3 \mapsto [2, 1, 3]$. The canonical form strips trailing zeros, so each monomial has a unique representation ($a^2 \mapsto [2]$ regardless of the ambient variable count), and the constant monomial is the empty vector.

Polynomials.
A finite map from canonical monomials to *nonzero* rational coefficients. Maintaining this normal form (no stored zeros, like terms combined) makes polynomial equality *structural*: $p = q$ iff their maps are identical.

Remark 3.1. Structural equality of normal forms is what makes the trusted core small: the checker never reasons about polynomials symbolically; it computes both sides of (1) into normal form and compares maps.

4 The trusted checker

The checker is the only component that must be correct. Given a target p and a candidate certificate $\big((c_i, q_i)\big)$ it performs the three exact steps in Algorithm 1.

```
check_sos (p, cert):
    if any c_i < 0 in cert     return false   # (1) nonnegative coefficients
    R := sum_i  c_i * (q_i * q_i)             # (2) exact rational expansion
    return  (p == R)                          # (3) equality of normal forms
```

Figure 1: The trusted checker `check_sos`.

Proposition 4.1 (Checker soundness)**.** *If* `check_sos`$(p, cert)$ *returns* `true`*, then* $p \geq 0$*.*

Proof. On returning `true`, step (1) guarantees $c_i \geq 0$ for all i, and steps (2)–(3) guarantee the polynomial identity $p = \sum_i c_i q_i^2$ holds in $\mathbb{Q}[\mathbf{x}]$ (equal normal forms are equal polynomials). Theorem 2.3 then gives $p \geq 0$. □

Proposition 4.1 is the entire trusted-computing-base argument: it depends only on exact rational arithmetic, exact polynomial multiplication and addition, and structural equality, and on nothing about how the certificate was found.

5 The prover

The prover is the untrusted search for a certificate. Its mathematical core is the Gram-matrix (or "Gram-matrix method") reformulation of the SOS condition.

5.1 The Gram-matrix reformulation

Fix a finite vector of monomials $\mathbf{z} = (z_1, \dots, z_r)^\top$, the *basis*. For a symmetric matrix $Q \in \mathbb{Q}^{r\times r}$ consider the quadratic form $\mathbf{z}^\top Q\,\mathbf{z} = \sum_{i,j} Q_{ij}\, z_i z_j$, a polynomial.

Lemma 5.1 (Gram representation)**.** *Let $\mathbf{z}$ be a monomial basis. A polynomial p is a sum of squares of polynomials in* $\operatorname{span}\mathbf{z}$ *if and only if there exists a symmetric positive semidefinite matrix $Q \succeq 0$ with $p = \mathbf{z}^\top Q\,\mathbf{z}$.*

Proof. ($\Leftarrow$) If $Q \succeq 0$, decompose $Q = \sum_k \lambda_k \mathbf{v}_k \mathbf{v}_k^\top$ with $\lambda_k \geq 0$ (e.g. a spectral decomposition). Then

$$\mathbf{z}^\top Q\,\mathbf{z} = \sum_k \lambda_k\, (\mathbf{v}_k^\top \mathbf{z})(\mathbf{z}^\top \mathbf{v}_k) = \sum_k \lambda_k\, (\mathbf{v}_k^\top \mathbf{z})^2,$$

an SOS with square bases $q_k = \mathbf{v}_k^\top \mathbf{z} \in \operatorname{span}\mathbf{z}$.

($\Rightarrow$) If $p = \sum_k \lambda_k q_k^2$ with $\lambda_k \geq 0$ and $q_k = \mathbf{v}_k^\top \mathbf{z}$, set $Q = \sum_k \lambda_k \mathbf{v}_k \mathbf{v}_k^\top \succeq 0$; then $\mathbf{z}^\top Q\,\mathbf{z} = \sum_k \lambda_k q_k^2 = p$. □

Matching coefficients turns $p = \mathbf{z}^\top Q\,\mathbf{z}$ into *linear* constraints on Q: grouping the products $z_i z_j$ by the monomial μ they equal,

$$\sum_{(i,j)\,:\, z_i z_j = \mu} Q_{ij} \;=\; \operatorname{coeff}_p(\mu) \qquad \text{for every monomial } \mu. \tag{2}$$

The solution set of (2) is an affine subspace of symmetric matrices; finding a positive semidefinite point in it is precisely a *semidefinite feasibility problem*. Its size and difficulty are governed by the choice of basis $\mathbf{z}$, which the Newton polytope fixes canonically.

5.2 Choosing the basis: the Newton polytope

Lemma 5.1 presupposes a basis $\mathbf{z}$, and the choice is consequential: a basis that is too small cannot represent the certificate at all, while one that is too large inflates the semidefinite program (2) and introduces spurious degrees of freedom. The *Newton polytope* determines the correct basis canonically.

Definition 5.2 (Newton polytope)**.** For $p = \sum_\alpha c_\alpha \mathbf{x}^\alpha$ the *Newton polytope* is the convex hull of the exponent vectors of its monomials,

$$\operatorname{New}(p) \;=\; \operatorname{conv}\{\, \alpha \in \mathbb{N}^n : c_\alpha \neq 0 \,\} \;\subseteq\; \mathbb{R}^n.$$

Theorem 5.3 (Reznick, 1978)**.** *If $p = \sum_i q_i^2$ is a sum of squares, then* $\operatorname{New}(q_i) \subseteq \frac{1}{2}\operatorname{New}(p)$ *for every i.*

Thus every square in *any* SOS decomposition of p is supported on the lattice points of the half-polytope, and it suffices to take the monomial basis

$$\mathbf{z} \;=\; \big\{\, \mathbf{x}^\beta : \beta \in \mathbb{N}^n,\; 2\beta \in \operatorname{New}(p) \,\big\}. \tag{3}$$

By Theorem 5.3 this basis is *complete* (it contains a valid basis whenever p is a sum of squares), and it is as small as that bound allows, which minimises the dimension of the Gram problem (2). It also specialises correctly: when $\deg p \leq 2$, (3) is exactly $\{1, x_1, \dots, x_n\}$, recovering the basis of Section 5.3.

Remark 5.4 (A cost-free necessary test)**.** A vertex α of $\operatorname{New}(p)$ can be produced only by the square of the single basis monomial $\mathbf{x}^{\alpha/2}$ (no cancellation is possible at an extreme point), so if p is a sum of squares then *every vertex of* $\operatorname{New}(p)$ *has even coordinates and positive coefficient.* This rejects many non-SOS inputs before any semidefinite computation is attempted.

5.3 Degree two: the unique Gram matrix

Take the basis $\mathbf{z} = (1, x_1, \ldots, x_n)^\top$. Every monomial of degree ≤ 2 is the product of *exactly one* unordered pair of basis elements: $1 = 1 \cdot 1$, $x_i = 1 \cdot x_i$, $x_i^2 = x_i \cdot x_i$, and $x_i x_j = x_i \cdot x_j$. Hence (2) *determines Q uniquely*:

$$Q_{00} = \operatorname{coeff}_p(1), \quad Q_{0i} = \tfrac{1}{2}\operatorname{coeff}_p(x_i), \quad Q_{ii} = \operatorname{coeff}_p(x_i^2), \quad Q_{ij} = \tfrac{1}{2}\operatorname{coeff}_p(x_i x_j)\ (i < j).$$

Proposition 5.5 (No gap in degree two)**.** *Write $p(\mathbf{x}) = \mathbf{x}^\top A\,\mathbf{x} + 2\,\mathbf{b}^\top \mathbf{x} + c$ and let $Q = \left(\begin{smallmatrix} c & \mathbf{b}^\top \\ \mathbf{b} & A \end{smallmatrix}\right)$ be the Gram matrix above. Then the following are equivalent:* (i) *$p \ge 0$;* (ii) *$Q \succeq 0$;* (iii) *p is a sum of squares.*

Proof. (ii)⇒(iii) is Lemma 5.1, and (iii)⇒(i) is Theorem 2.3. For (i)⇒(ii): for any $\mathbf{x}$ and $t \neq 0$, $(\mathbf{x}, t)\, Q\, (\mathbf{x}, t)^\top = t^2 p(\mathbf{x}/t) \ge 0$; taking $t \to 0$ and using continuity gives $(\mathbf{x}, t) Q (\mathbf{x}, t)^\top \ge 0$ for all $(\mathbf{x}, t)$, i.e. $Q \succeq 0$. □

Thus for quadratic targets the prover is *complete*: it succeeds exactly when the inequality is true.

5.4 Exact extraction: rational $LDL^\top$

It remains to turn a rational PSD Q into an explicit rational SOS. This is done by symmetric Gaussian elimination.

Lemma 5.6 (Zero pivots of a PSD matrix)**.** *If $Q \succeq 0$ and $Q_{kk} = 0$, then $Q_{kj} = 0$ for all j.*

Proof. The principal 2×2 submatrix on indices $\{k, j\}$ is $\left(\begin{smallmatrix} 0 & Q_{kj} \\ Q_{kj} & Q_{jj} \end{smallmatrix}\right)$ and must be PSD; its determinant $-Q_{kj}^2 \ge 0$ forces $Q_{kj} = 0$. □

Theorem 5.7 (Rational PSD ⇒ rational SOS)**.** *Let $Q \in \mathbb{Q}^{r \times r}$ be symmetric and PSD. The factorisation $Q = LDL^\top$ with L unit lower-triangular and D diagonal, computed by*

$$D_k = Q_{kk} - \sum_{j<k} L_{kj}^2 D_j, \qquad L_{ik} = \frac{1}{D_k}\Big(Q_{ik} - \sum_{j<k} L_{ij} L_{kj} D_j\Big)\ (i > k), \tag{4}$$

has $L, D \in \mathbb{Q}$ and $D_k \ge 0$ for all k. Consequently

$$p = \mathbf{z}^\top Q\, \mathbf{z} = \sum_{k : D_k > 0} D_k\, q_k^2, \qquad q_k = (L^\top \mathbf{z})_k = \sum_j L_{jk}\, z_j \in \mathbb{Q}[\mathbf{x}],$$

is an exact rational SOS certificate for p.

Proof. The quantity D_k is the k-th pivot, equal to the Schur complement diagonal entry; for a PSD matrix all pivots are ≥ 0. If $D_k > 0$, (4) is a well-defined rational division. If $D_k = 0$, then by Lemma 5.6 (applied to the Schur complement, itself PSD) the remaining entries of that column vanish, so no division occurs and $L_{ik} = 0$; no pivoting is needed. All operations are in $\mathbb{Q}$, so L, D are rational. Finally $Q = LDL^\top$ gives $\mathbf{z}^\top Q \mathbf{z} = (L^\top \mathbf{z})^\top D (L^\top \mathbf{z}) = \sum_k D_k (L^\top \mathbf{z})_k^2$, and the terms with $D_k = 0$ drop. □

Corollary 5.8. *A symmetric rational positive semidefinite matrix is a sum of rational squares. Over $\mathbb{Q}$, for the Gram problem, "PSD" and "SOS-decomposable" coincide, and the decomposition is obtained exactly.*

Example 5.9 (Worked degree-two case)**.** For $p = a^2 - ab + b^2$ and $\mathbf{z} = (a, b)^\top$, $Q = \begin{pmatrix} 1 & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{pmatrix}$. Then (4) gives $D_1 = 1$, $L_{21} = -\frac{1}{2}$, $D_2 = 1 - \frac{1}{4} = \frac{3}{4}$, so

$$a^2 - ab + b^2 \;=\; 1 \cdot \big(a - \tfrac{1}{2} b\big)^2 + \tfrac{3}{4}\, b^2.$$

5.5 Higher degree and the semidefinite regime

For $\deg p > 2$ the basis is the Newton-polytope basis (3) of Section 5.2. (The present implementation uses computationally cheaper subsets (single-variable "square roots" of p's perfect-square monomials and, for homogeneous targets, all degree-d monomials), which are sound but can be incomplete; adopting the full Newton basis is part of the prover's planned form, Section 11.) With the basis fixed, two situations occur.

Unique Gram. If every product monomial in (2) arises from a single pair, Q is again determined, and Theorem 5.7 finishes the job. For instance $a^4 + b^4 \geq 2a^2b^2$ uses $\mathbf{z} = (a^2, b^2)^\top$ and $Q = \left(\begin{smallmatrix} 1 & -1 \\ -1 & 1 \end{smallmatrix}\right)$, giving $(a^2 - b^2)^2$.

Under-determined Gram (an SDP). When a monomial is the product of several pairs (e.g. $a^2b^2 = (a^2)(b^2) = (ab)(ab)$), the constraints (2) leave free parameters, and one must *choose* a PSD member of the affine family. This is a genuine semidefinite program, and the prover meets it two ways. Over up to two free entries it runs a bounded rational grid search; beyond that it emits the program for an external numerical solver and rounds the returned matrix to an exact rational PSD one (Section 11). Either way every candidate satisfies $\mathbf{z}^\top Q\,\mathbf{z} = p$ by construction, so any PSD candidate yields a valid certificate, and the trusted checker verifies it. Targets with no SOS representation at all (Example 2.4) correctly yield NO_CERT_FOUND.

Soundness is independent of all of this. Regardless of which basis, which grid point, or which numerical solver produces Q, the certificate is re-checked by `check_sos`. The search is untrusted; only Proposition 4.1 is relied upon.

6 Clearing denominators

Many olympiad inequalities are stated with fractions. The parser accepts division between expressions, and the normaliser reduces a rational-function claim to a polynomial one, soundly.

Proposition 6.1 (Denominator clearing)**.** *Let A, B be rational functions and write $A - B = N/D$ with $N, D \in \mathbb{Q}[\mathbf{x}]$ and $D \not\equiv 0$. Then, at every point where $D \neq 0$,*

$$A - B \geq 0 \iff N \cdot D \geq 0.$$

Hence proving $p := N \cdot D \geq 0$ on all of $\mathbb{R}^n$ establishes $A \geq B$ wherever it is defined.

Proof. Where $D \neq 0$ we have $D^2 > 0$, and N/D and $N \cdot D = (N/D)\,D^2$ have the same sign. Thus $N/D \geq 0 \iff ND \geq 0$. The claim follows, and note that no assumption is made on the sign of D. □

The construction proceeds by evaluating each subexpression to a numerator–denominator pair (N, D) and combining fractions; when the input contains no division, $D \equiv 1$ and $p = A - B$ as before, so the polynomial case is recovered exactly.

7 Constrained inequalities: a fragment of the Positivstellensatz

Sum of squares proves an inequality *everywhere* on $\mathbb{R}^n$. But a great many olympiad inequalities are not true everywhere; they hold only on a region, under hypotheses like $a, b, c \geq 0$, or on the surface $abc = 1$, or for the side lengths of a triangle. Off the feasible set such a target may well dip below zero, so a plain SOS decomposition cannot exist, and asking for one is the wrong question. What we want are certificates that are allowed to *use* the hypotheses.

The right generalisation is a fragment of the *Positivstellensatz*. Say we want $p \geq 0$ on the set carved out by inequality constraints $g_1, \ldots, g_m \geq 0$ and equality constraints $h_1, \ldots, h_k = 0$. A certificate is an identity

$$p \;=\; \sigma_0 \;+\; \sum_{i=1}^{m} \sigma_i \, g_i \;+\; \sum_{j=1}^{k} \lambda_j \, h_j, \tag{5}$$

in which every σ_i is a sum of squares and every λ_j is an arbitrary polynomial. The soundness argument is as short as the one for plain SOS, and worth writing out because it says exactly what each piece is for.

Theorem 7.1 (Soundness of Positivstellensatz certificates)**.** *Suppose p satisfies (5) with each σ_i a sum of squares. Then $p(\mathbf{x}) \geq 0$ at every point $\mathbf{x}$ with $g_i(\mathbf{x}) \geq 0$ for all i and $h_j(\mathbf{x}) = 0$ for all j.*

Proof. Fix such an $\mathbf{x}$. Then $\sigma_0(\mathbf{x}) \geq 0$; each $\sigma_i(\mathbf{x}) \geq 0$ (a sum of squares) while $g_i(\mathbf{x}) \geq 0$, so $\sigma_i(\mathbf{x})\, g_i(\mathbf{x}) \geq 0$; and each $\lambda_j(\mathbf{x})\, h_j(\mathbf{x}) = \lambda_j(\mathbf{x}) \cdot 0 = 0$. The right-hand side of (5) is thus a sum of nonnegative terms, giving $p(\mathbf{x}) \geq 0$. □

The two kinds of constraint play genuinely different roles, and the proof makes the difference visible. An inequality constraint contributes a nonnegative term, a square times something nonnegative, so its multiplier must itself be a sum of squares. An equality constraint contributes *nothing*: it vanishes on the feasible set, so its multiplier λ_j is unconstrained, free to be any polynomial of any sign. That asymmetry is mirrored exactly in the checker.

7.1 The checker, extended

The trusted core gains one function, `check_constrained`, that verifies (5). It is handed the target p, the declared hypotheses, and a certificate whose terms are tagged by kind, and it confirms three things: every sum-of-squares part (σ_0 and each σ_i) has nonnegative coefficients; every polynomial being scaled really is one of the declared hypotheses *of the matching kind* (a g_i for an inequality term, an h_j for an equality term), compared by structural equality; and the expansion of the right-hand side equals p exactly. The membership test is the subtle one, and it is not optional: without it a "certificate" could scale a polynomial that is negative somewhere and is not a stated assumption, and the whole thing would be unsound. With it, Theorem 7.1 applies verbatim, and the trusted base is still nothing but exact arithmetic and structural equality.

One further shape earns its own term. Schmüdgen's theorem reminds us that we sometimes need *products* of the inequality constraints: from $a \geq 0$ and $b \geq 0$ we get $ab \geq 0$, but ab is neither a hypothesis on its own nor a sum of squares. So the format also admits terms $\sigma \cdot g_{i_1} \cdots g_{i_r}$: a square times a product of declared nonnegative hypotheses. The checker validates each factor against the hypothesis list, and (square) × (product of nonnegatives) ≥ 0 closes the argument.

7.2 Finding the multipliers

Searching for the σ_i and λ_j is, in full generality, a semidefinite program again, now with one Gram matrix per constraint. The prover does not solve that yet. Instead it tries three cheap, exact strategies in turn, each one re-checked by `check_constrained` before anything is believed:

- **Constant multipliers.** Let the σ_i and λ_j range over a small grid of rational constants, subtract $\sum c_i g_i + \sum d_j h_j$ from p, and hand the remainder to the ordinary SOS prover to play the role of σ_0. This alone dispatches, say, $a^2 + b^2 \geq 2$ given $ab = 1$, through $a^2 + b^2 - 2 = (a - b)^2 + 2(ab - 1)$.

- **Polynomial multipliers on equalities.** Since an equality multiplier may be *any* polynomial, we can compute one directly by exact polynomial division: reduce p modulo h_j to get $p = \lambda_j h_j + r$, then try to prove the remainder r. Division is taken with respect to the lexicographic monomial order (a genuine, multiplication-compatible term order, so the reduction always terminates), and it turns $a^3 \geq b^3$ given $a = b$ into $a^3 - b^3 = (a^2 + ab + b^2)(a - b)$.
- **Products of hypotheses.** For each pair of nonnegative hypotheses g_i, g_j and a constant c, subtract $c\, g_i g_j$ and prove the remainder. This is what settles $ab \geq 0$ given $a, b \geq 0$.

These are deliberately modest. They miss anything that needs a truly non-constant square multiplier on an inequality, or a product of three or more hypotheses, and the general case is the same SDP story as the unconstrained prover (Section 11). But they already reach a satisfying spread of constrained problems, and, like every other thing the search does, they cannot be wrong: a proposed certificate is believed only once the trusted checker has re-derived the identity (5) for itself.

8 System design

8.1 Pipeline and the trust boundary

Figure 2 shows the flow from an input string to a rendered proof. The defining invariant is the one-way valve at the checker: the status PROVED is emitted *iff* the trusted checker accepts.

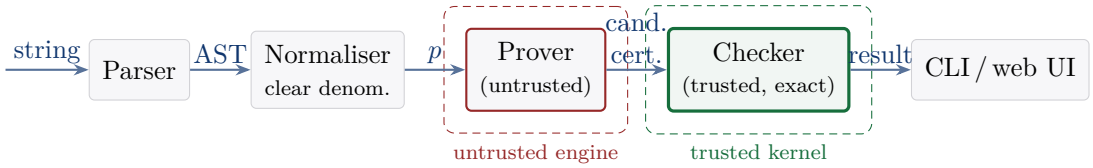

Figure 2: The A ≥ B pipeline. Only the checker is trusted; a fault in the prover can cause a missed proof, never a false one.

8.2 Module layering

The library is layered so that each module depends only on those below it, which keeps the trusted checker's dependencies minimal:

$$\begin{aligned}\mathsf{Rational} &\to \mathsf{Monomial} \to \mathsf{Polynomial} \to \{\mathsf{Ast},\ \mathsf{Pretty}\} \\ &\to \{\mathsf{Parser},\ \mathsf{Normalizer},\ \mathsf{Certificate},\ \mathsf{Constrained}\} \to \mathsf{Checker} \to \mathsf{Prover} \to \mathsf{Sdp} \\ &\to \mathsf{Proof\_result} \to \{\text{CLI},\ \text{web server}\}.\end{aligned}$$

Rational is the sole point of contact with the bignum backend, which localises the exact-arithmetic dependency (and eases a future pure-rational or formally verified reimplementation).

8.3 Structured results

Everything downstream of the checker consumes one record. Proof_result runs the pipeline of Figure 2 on a claim string and returns the run as data: the variable context and hypotheses, the reduced target, the final status, the accepted certificate (if any) together with the search route that produced it, warnings when the hypotheses are provably contradictory (so a "proof" would hold only vacuously), and a chronological trace of the stages that actually ran, each tagged trusted or untrusted and, on request, timed. The status PROVED is assigned in exactly one place, after `check_sos` or `check_constrained` accepts, so every interface inherits the trust

boundary instead of re-implementing it; a front end cannot report a proof the record does not contain.

The record also fixes the story told about the numerical route. The hook that reaches the external solver is a parameter: a caller may supply a function that solves the emitted Gram program numerically, and its output re-enters the pipeline as a candidate like any other, through rational reconstruction and the exact check. The trace then records the miss of the exact search, the solver's reported status, and the reconstruction, so "how was this proved?" has a faithful answer even when floating point was involved in the middle.

8.4 The command line

The CLI exposes `prove "<inequality>"` (side conditions attach with `given`, e.g. `a*b >= 0 given a >= 0, b >= 0`); `check <cert.json>` (verify a stored certificate, plain or constrained); `lean "<inequality>"` (prove, then emit a machine-checkable Lean proof, Section 10); and the `sdp-emit`/`sdp-check` pair driven by the numerical pipeline. Every run reports one of four statuses with matching exit codes: PROVED, NO_CERT_FOUND, INVALID_INPUT, CHECK_FAILED.

8.5 The local web interface

`a-geq-b-web` serves the same prover in a browser. It is deliberately small: a hand-written HTTP/1.1 subset over OCaml's `Unix` sockets, bound to the loopback interface, serving a static page and two JSON endpoints (`POST /api/prove`, the serialised Proof_result; `POST /api/lean`, the exported theorem of Section 10). No web framework is involved, and no mathematics is duplicated in the browser: the page renders the record, so the project's dependency surface stays at the bignum and JSON libraries.

The page presents an accepted certificate as the identity the checker verified, and every result offers the recorded proof path with its trust tags, which makes the boundary of Figure 2 visible on each proof rather than only in documentation. When the Python solver's virtualenv is present, the server installs the SDP hook of Section 8.3 and browser proofs of harder targets go through the numerical route live; when it is absent, the trace says so. Failure states are worded to match the semantics: NO_CERT_FOUND is always accompanied by the statement that it is not a disproof.

9 Validation

The proofs above argue that the system is sound; a benchmark corpus is how I keep myself honest that it is also *useful*, and stays that way as the code changes. It holds 119 inequalities, each tagged with a category and the status the system ought to return (Table 1), drawn from classical results (AM-GM, Cauchy-Schwarz, Schur), named competition problems (IMO, APMO, and several national olympiads), and a standard olympiad reference, together with a handful of deliberate soundness traps.

Category	Meaning	Expected status
`in_scope_sos`	nonnegative on $\mathbb{R}^n$, SOS over $\mathbb{Q}$	PROVED
`out_of_scope_v1`	true only under side constraints	NO_CERT_FOUND
`not_sos`	nonnegative but not SOS (Motzkin, Choi-Lam)	NO_CERT_FOUND
`false`	deliberately false	NO_CERT_FOUND

Table 1: Corpus categories and the status each should elicit.

Three independent harnesses guard the corpus: one runs every SOS certificate through the trusted checker; one is an *independent* numeric oracle that random-samples each claim on

its domain (implemented in a different language, so an error in the core cannot mask a bad entry); and one enforces the *soundness gate*: no `false`, constrained, or `not_sos` entry may ever be proved. The gate is not decorative: it flagged a problem originally recorded as requiring positivity, whose difference is in fact the perfect square

$$(a+b)^2(a+c)^2 - 4abc(a+b+c) = (a^2+ab+ac-bc)^2,$$

prompting its reclassification as an unconditional SOS identity.

The unit suites complement the corpus with 139 cases: algebraic laws and canonicalisation, the parser (including denominator clearing and rejection of malformed input), the checker against adversarial certificates (negative weights, wrong expansions, missing and extra terms, terms scaling an undeclared hypothesis), prover soundness on false and non-SOS targets, Lean export, SDP rounding, and the structured result layer, including the test that a numerically wrong Gram matrix supplied through the SDP hook ends in NO_CERT_FOUND, never PROVED. A browser suite (Playwright) then drives the web interface against the real server and asserts the same semantics at the surface: what may be labelled PROVED, that NO_CERT_FOUND is never presented as falsity, that the rendered proof path matches the route the record reports, and that the Lean view never claims a kernel check that did not happen.

10 Formal verification in Lean 4

Proposition 4.1 is the hinge the entire design turns on, so it is the one claim most worth putting beyond doubt. Two mechanisations in Lean 4, both complete, now do that, from two different angles.

10.1 A verified checker

The first reimplements the checker over Mathlib's `MvPolynomial (Fin` n`)` $\mathbb{Q}$ and proves its soundness as a theorem the Lean kernel checks:

$$\texttt{checkSOS}\ p\ \mathit{cert} = \texttt{true} \implies \forall\, \mathbf{x} : \mathrm{Fin}\, n \to \mathbb{R},\quad 0 \le \mathrm{aeval}\, \mathbf{x}\, p.$$

This is Proposition 4.1, word for word, and its proof is the formal shadow of Theorem 2.3: once `checkSOS` has forced the identity $p = \sum_i c_i q_i^2$, nonnegativity falls straight out of `sq_nonneg`, `mul_nonneg`, and `List.sum_nonneg`. The proof is complete and axiom-clean: `#print axioms` reports only the three standard Mathlib axioms (`propext`, `Classical.choice`, `Quot.sound`) and not a single `sorry`. And because Lean 4 compiles to native code, this is not merely a proof *about* a checker; it is a checker that runs, and could stand as an independent oracle beside the OCaml one.

10.2 One machine-checked proof per inequality

The second goes further and makes each individual *proof* machine-checked. The command `a-geq-b lean "<inequality>"` runs the search, re-checks the certificate, and emits a self-contained Lean file: a theorem $0 \le p$ over real variables whose proof rewrites p into its sum-of-squares form with `ring` (the identity the checker has already verified) and then closes it with `positivity`. Feeding that file to Lean has its kernel certify the result, so an $A \ge B$ proof becomes a theorem in a general-purpose proof assistant. It is untrusted output in the best possible sense: a bogus certificate can only ever produce a Lean file that fails to compile, never a false theorem.

11 Roadmap

The design is in place: the trusted checker and its Lean-verified twin (Section 10), the exact SOS prover, denominator clearing, the constrained search of Section 7, and the numerical semidefinite prover described below, which closes the last in-scope corpus targets. What remains only sharpens an already-working prover, and, as always, keeps the checker the sole authority.

The numerical semidefinite prover. For a target whose Gram matrix has more freedom than the bounded grid search can explore, the exact core emits the Gram program (2) as JSON and an external numerical solver (cvxpy's CLARABEL interior-point method) returns an approximate positive semidefinite Q. The core then rounds Q back to exact rationals: it fixes the free entries at a small common denominator and completes the dependent entries so that $\mathbf{z}^\top Q\,\mathbf{z} = p$ holds identically, so the recovered matrix is an exact rational Gram matrix of p whenever the rounding lands inside the PSD cone. An exact $LDL^\top$ then reads off the certificate, which the trusted checker verifies as usual. The numerical code is quarantined cleanly behind the trust boundary: a wrong solution rounds to a matrix that is not PSD, or to a rejected certificate, never a false proof. With this path the prover closes all 63 in-scope sum-of-squares targets of the corpus, while the not-SOS traps (Motzkin, Choi-Lam) and the false claims stay unproved.

What remains is optional refinement. The constrained search of Section 7 wants the same machinery for its hardest cases: a non-constant square multiplier σ_i on an inequality is itself a Gram problem, as is a product of three or more hypotheses. Building the basis by the Newton-polytope reduction (3) rather than the full degree-d set would shrink the programs further. Both improve a prover that already works rather than fill a gap.

The system now realises a principled three-language architecture: exact symbolic computation and the trusted core in a typed functional language (OCaml), numerical semidefinite programming (untrusted) in the scientific-computing ecosystem (Python), and a machine-checked proof of the kernel in a proof assistant (Lean 4). Each language does what it is best at, and only the smallest of them has to be believed.

References (selected)

[1] D. Hilbert. *Über die Darstellung definiter Formen als Summe von Formenquadraten.* Math. Ann. **32** (1888), 342-350.

[2] T. S. Motzkin. *The arithmetic-geometric inequality.* In *Inequalities* (1967), 205-224.

[3] M. D. Choi, T. Y. Lam. *Extremal positive semidefinite forms.* Math. Ann. **231** (1977), 1-18.

[4] B. Reznick. *Extremal PSD forms with few terms.* Duke Math. J. **45** (1978), 363-374.

[5] V. Powers, T. Wörmann. *An algorithm for sums of squares of real polynomials.* J. Pure Appl. Algebra **127** (1998), 99-104.

[6] P. A. Parrilo. *Semidefinite programming relaxations for semialgebraic problems.* Math. Program. **96** (2003), 293-320.

[7] H. Peyrl, P. A. Parrilo. *Computing sum of squares decompositions with rational coefficients.* Theoret. Comput. Sci. **409** (2008), 269-281.

[8] J. B. Lasserre. *Global optimization with polynomials and the problem of moments.* SIAM J. Optim. **11** (2001), 796-817.

[9] R. B. Manfrino, J. A. G. Ortega, R. V. Delgado. *Inequalities: A Mathematical Olympiad Approach.* Birkhäuser, 2009.